COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Reyes, Eli Justin L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO B.2

INPUT

main.cpp > main()

```
#include <iostream>    #include errors detected. Please upda
#include "queuearray.h"

int main() {
    qarray<int> q(5);
    q.Enqueue(1);
    q.Enqueue(2);
    q.Enqueue(3);

    qarray<int> q2 = q;   // copy constructor
    q2.Dequeue();

    std::cout << "q size: " << q.Size() << std::endl;
    std::cout << "q2 size: " << q2.Size() << std::endl;

    qarray<int> q3(2);
    q3 = q;               // copy assignment

    std::cout << "q3 size: " << q3.Size() << std::endl;
    std::cout << "q3 front: " << q3.Front() << std::endl;

    return 0;
}
```

```cpp
#ifndef SUP_ACT_5_QUEUEARRAY_H
#define SUP_ACT_5_QUEUEARRAY_H

#include <iostream>    #include errors detected. Please

template <typename T>
class qarray {
private:
    T* q_array = nullptr;
    size_t q_capacity = 8;
    size_t q_size = 0;
    int q_front = -1;
    int q_back = -1;

public:
    qarray(size_t capacity);
    bool Empty();
    bool Full();

    size_t Size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T newElement);
    void Dequeue();

    ~qarray();

    qarray(const qarray& other);
    qarray& operator=(const qarray& other);
};

template <typename T>
qarray<T>::qarray(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

template <typename T>
qarray<T>::~qarray() {
    delete [] q_array;
}
```

```cpp
template <typename T>
qarray<T>::~qarray() {
    delete [] q_array;
}

template <typename T>
size_t qarray<T>::Size() {
    return q_size;
}

template <typename T>
bool qarray<T>::Empty() {
    return q_size == 0;
}

template <typename T>
bool qarray<T>::Full() {
    return q_size == q_capacity;
}

template <typename T>
void qarray<T>::Clear() {
    q_size = 0;
    q_front = -1;
    q_back = -1;
}

template <typename T>
T qarray<T>::Front() {
    if (Empty()) {
        std::cout<<"Q is empty\n";
        return T{};
    }
    return q_array[q_front];
}

template <typename T>
T qarray<T>::Back() {
    if (Empty()) {
        std::cout<<"Q is empty\n";
        return T{};
    }
    return q_array[q_back];
}
```

```cpp
template <typename T>
void qarray<T>::Enqueue(T newElement) {
    if (Full()) {
        std::cout<<"Q is full\n";
        return;
    }

    q_back = (q_back + 1) % q_capacity;
    q_array[q_back] = newElement;
    q_size++;
}

template <typename T>
void qarray<T>::Dequeue() {
    if (Empty()) {
        std::cout<<"Q is empty\n";
        return;
    }

    q_front = (q_front + 1) % q_capacity;
    q_size--;
}

template <typename T>
qarray<T>::qarray(const qarray& other) {
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }
}

template <typename T>
qarray<T>& qarray<T>::operator=(const qarray& other) {
    if (this == &other) {
        return *this;
    }

    delete [] q_array;

    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
```

```
132     q_capacity = other.q_capacity;
133     q_size = other.q_size;
134     q_front = other.q_front;
135     q_back = other.q_back;
136     q_array = new T[q_capacity];
137     for (size_t i = 0; i < q_capacity; i++) {
138         q_array[i] = other.q_array[i];
139     }
140
141     return *this;
142 }
143
144 #endif //SUP_ACT_5_QUEUEARRAY_H
```

OUTPUT

```
q size: 3
q2 size: 2
q3 size: 3
q3 front: 1
```

Code Analysis / Conclusion	Program to build a queue data structure using circular arrays as qarray, where qarray is a template class with an array allocated to its own memory. This program also tests whether the deep copy has been done in the copy constructor and copy assignment operator of qarray. The compiler's generated functions would have just copied the pointer q_array and therefore two separate objects will share the same memory causing problems like double free during the destructor. Both constructors have been built explicitly in order to allocate an array in their own memory and also perform self-assignment check and deallocation before the operation. As a result of testing, we can see that changing any one of the q objects does not have any effect on q because each object has its own separate memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

INPUT

main.cpp

```
#include "printerqueue.h"    #include errors de

int main() {
    Printer<Job> printer;

    printer.addJob(Job(1, "Adrian", 5));
    printer.addJob(Job(2, "Carl", 12));
    printer.addJob(Job(3, "Bien", 3));
    printer.processAll();

    printer.addJob(Job(4, "Roy", 8));
    printer.addJob(Job(5, "Twain", 1));
    printer.processNext();
    printer.addJob(Job(6, "Eli", 20));
    printer.processAll();

    printer.processNext();

    return 0;
}
```

printerqueue.h

```cpp
#ifndef PRINTER_QUEUE_H
#define PRINTER_QUEUE_H

#include <iostream>    #include errors detected. Please upd
#include <string>

class Job {
public:
    int id;
    std::string user;
    int pages;

    Job(int id, std::string user, int pages) : id(id), user
};

template <typename T>
struct Node {
    T data;
    Node* next;

    Node(T data);
};

template <typename T>
Node<T>::Node(T data) : data(data), next(nullptr) {}

template <typename T>
class Printer {
private:
    Node<T>* front = nullptr;
    Node<T>* rear = nullptr;
    int count = 0;

public:
    ~Printer();

    bool isEmpty();
    int pending();
    void addJob(T job);
    bool processNext();
    void processAll();
};
```

```cpp
template <typename T>
Printer<T>::~Printer() {
    while (front) {
        Node<T>* temp = front;
        front = front->next;
        delete temp;
    }
}

template <typename T>
bool Printer<T>::isEmpty() {
    return front == nullptr;
}

template <typename T>
int Printer<T>::pending() {
    return count;
}

template <typename T>
void Printer<T>::addJob(T job) {
    Node<T>* node = new Node<T>(job);
    if (!rear) {
        front = rear = node;
    } else {
        rear->next = node;
        rear = node;
    }
    count++;
    std::cout << "Added: " << job.id << " " << job.user << " (" << job.pages << " pages)\n";
}

template <typename T>
bool Printer<T>::processNext() {
    if (isEmpty()) {
        std::cout << "Queue empty.\n";
        return false;
    }

```

```cpp
template <typename T>
bool Printer<T>::processNext() {
    if (isEmpty()) {
        std::cout << "Queue empty.\n";
        return false;
    }

    Node<T>* temp = front;
    T job = front->data;
    front = front->next;
    if (!front) rear = nullptr;
    delete temp;
    count--;

    std::cout << "Printed: " << job.id << " " << job.user << " (" << job.pages << " pages)\n";
    return true;
}

template <typename T>
void Printer<T>::processAll() {
    while (!isEmpty()) {
        processNext();
    }
}

#endif
```

OUTPUT

```
Added: 1 Adrian (5 pages)
Added: 2 Carl (12 pages)
Added: 3 Bien (3 pages)
Printed: 1 Adrian (5 pages)
Printed: 2 Carl (12 pages)
Printed: 3 Bien (3 pages)
Added: 4 Roy (8 pages)
Added: 5 Twain (1 pages)
Printed: 4 Roy (8 pages)
Added: 6 Eli (20 pages)
Printed: 5 Twain (1 pages)
Printed: 6 Eli (20 pages)
Queue empty.
```

Code Analysis / Conclusion	The terminal output confirms my FIFO queue works correctly: jobs process strictly in insertion order across multi-item batches and mid-queue additions, while the isEmpty() guard safely handles empty-queue calls. Also that red banner at the top isn't an error, probably a bug due to my extension (Error Lens) not a logic bug proven by gdb.exe executing below it. I encountered a bunch of logic errors earlier in processNext(), they were probably caused by common issues like deleting a node before reading its data or failing to set rear = nullptr when removing the final node and I couldn't figure it out.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error checking**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I couldn't figure out what was causing the error for processNext(), it kept looping back the queue and would just break. I asked Gemini what was causing the error.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.